

D2DC Challenge

REPORT

Submitted by

MODEL TINKERERS

1. KARUKKUVEL RAJ D

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **INCOSE India** and the organizers of the challenge for providing this platform to explore and implement Model-Based Systems Engineering (MBSE) principles using the Arcadia method and Capella.

A special note of thanks to **Nikhil Joshi** for his dedicated efforts in coordinating the challenge and providing the necessary guidance to navigate the complexities of the problem statement. His support was instrumental in the successful development of this architecture.

I am also deeply grateful to **Dr. Yogananda Jeppu** for his invaluable personal inputs and mentorship. His insights into systems engineering and safety-critical architectures provided me with the technical depth required to refine the logical and physical transitions of this system.

Finally, I would like to thank all the mentors and peers who provided feedback during the various stages of this project

It is worth mentioning about my teammates, friends and colleagues, for extending their kind help whenever the necessity arose. I thank one and all who have directly or indirectly helped me in making this project a great success.

1.	Introduction	iv
1.1	Background and Motivation	iv
1.2	Objective of the Study	iv
1.3	Scope and Assumptions.....	v
1.4	Methodology.....	v
1.5	Report Organization	vi
2.	Operational Analysis	vii
2.1	Operation Capabilities	vii
2.2	Operational Architecture	viii
3.	System Analysis	ix
3.1	Requirements	ix
3.2	System Architecture	ix
4.	Logical Architecture.....	xii
5.	Physical Architecture	xv
6.	Conclusion and Future Work	xvii

1. Introduction

The Nearby Driver-to-Driver Communication System is a modelling challenge defined by the INCOSE India MBSE Working Group. The challenge consists of developing and describing a system that enables direct communication between nearby drivers in order to support real-time coordination during on-road situations.

1.1 Background and Motivation

With the increasing density of road traffic and the coexistence of diverse vehicle types, effective communication between drivers has become a key enabler for safety, efficiency, and cooperative maneuvering. Driver-to-Driver Communication (D2DC) systems aim to support drivers by sharing intent, warnings, and maneuver-related information in real time, thereby reducing ambiguity and improving situational awareness.

This project addresses the conceptual design of a Driver-to-Driver Communication System using a Model-Based Systems Engineering (MBSE) approach. The focus is on defining system behavior, responsibilities, and architecture in a structured and traceable manner rather than implementation-specific solutions.

1.2 Objective of the Study

The objective of this work is to develop a consistent system architecture for a Driver-to-Driver Communication system by applying the Arcadia methodology using Capella. The study aims to:

- Capture stakeholder needs and system expectations
- Define system-level functions and interactions
- Structure logical responsibilities without introducing implementation bias
- Allocate logical elements to a physical architecture

- Maintain traceability across architectural layers

1.3 Scope and Assumptions

This work focuses on system-level design and does not address detailed software algorithms, hardware schematics, or communication protocol specifications. The system is analyzed assuming:

- Drivers interact with the system via Human–Machine Interfaces (HMI)
- Communication occurs between vehicles through abstracted communication mechanisms
- Environmental sensing and road conditions are considered at an abstract level
- Cybersecurity, regulatory compliance, and performance constraints are treated as non-functional considerations

1.4 Methodology

The development follows the Arcadia architectural framework, progressing through:

- Operational Analysis (OA)
- System Analysis (SA)
- Logical Architecture (LA)
- Physical Architecture (PA)

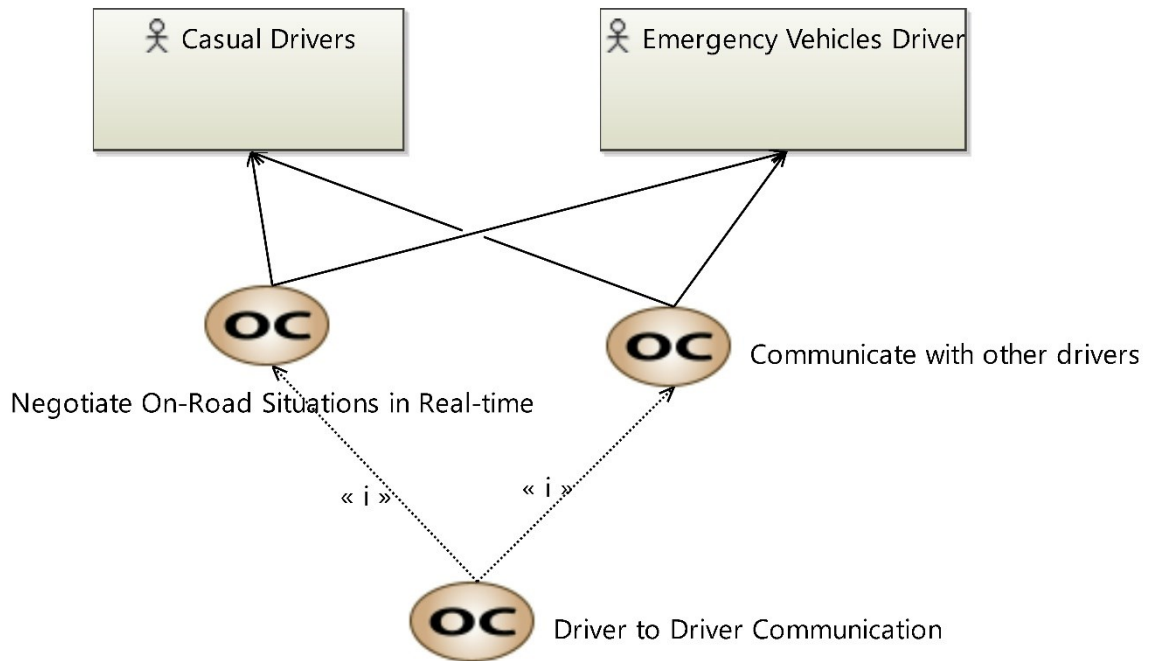
Each layer refines the previous one while preserving traceability and architectural consistency. Capella is used as the primary modelling tool to represent functions, components, exchanges, and scenarios.

1.5 Report Organization

This report is structured to reflect the Arcadia methodology. Each section presents the results of a specific architectural layer, followed by justification and traceability to ensure coherence across the system lifecycle.

2. Operational Analysis

2.1 Operation Capabilities

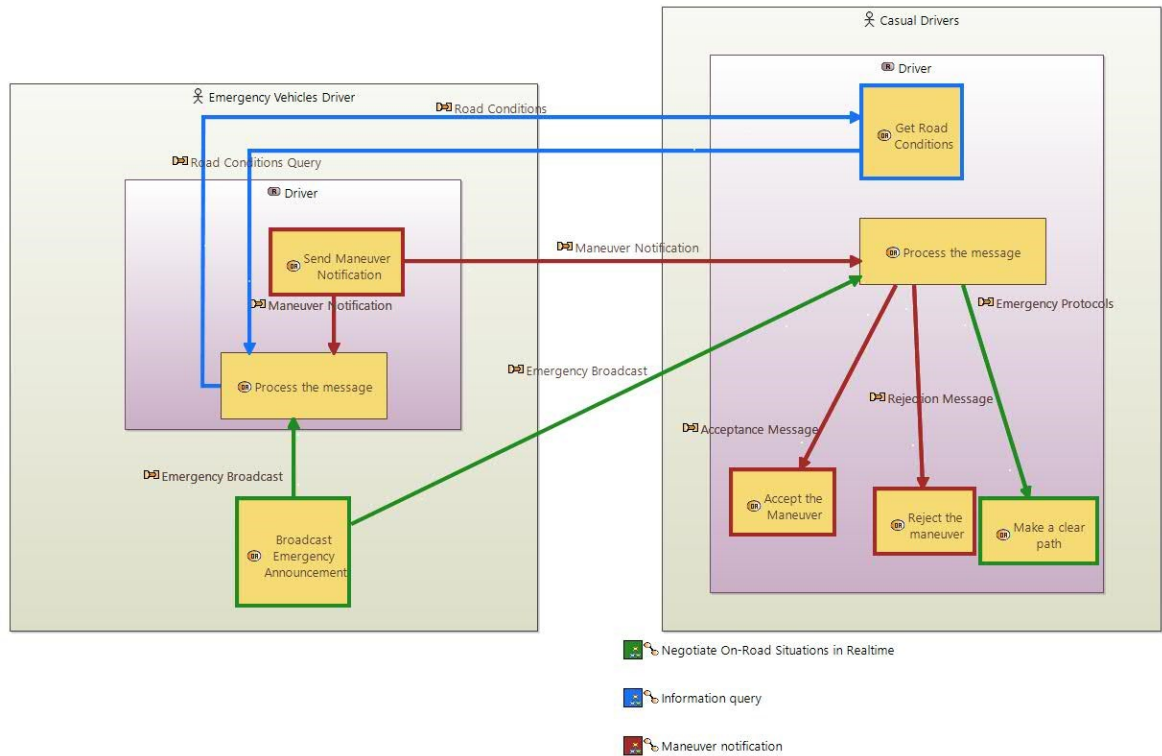


This diagram shows the main operational capabilities of the Driver-to-Driver Communication system. Casual Drivers and Emergency Vehicle Drivers are the primary actors interacting with the system.

The central capability, Driver to Driver Communication, enables drivers to exchange information with nearby vehicles. This capability includes Communicate with Other Drivers, which covers basic message exchange, and Negotiate On-Road Situations in Real-time, which supports collaborative handling of dynamic traffic situations.

These capabilities define what the system must enable at an operational level and provide the basis for further functional and architectural development.

2.2 Operational Architecture



This operational architecture illustrates how Casual Drivers and Emergency Vehicle Drivers interact during on-road situations. Emergency vehicle drivers initiate maneuver notifications and emergency broadcasts, while both driver types can query and share road condition information.

Incoming messages are processed by the receiving driver, leading to decisions such as accepting or rejecting a maneuver request, or taking actions like clearing a path in response to emergency protocols. The architecture captures the flow of information and decisions that enable real-time coordination between drivers in normal and emergency scenarios.

3. System Analysis

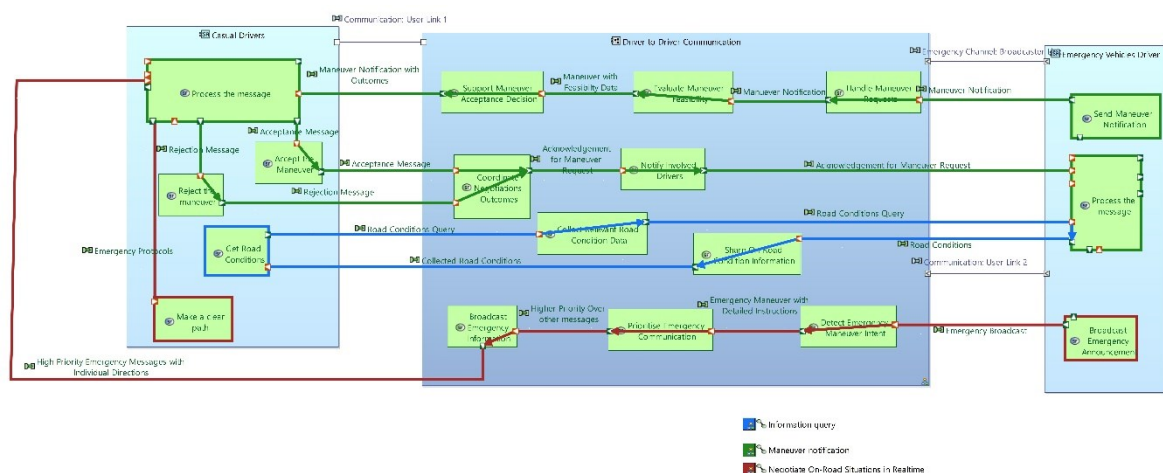
3.1 Requirements

The System User Requirements (SUR), System Functional Requirements (SFR), and System Non-Functional Requirements (SNFR) were derived from the operational analysis and identified operational needs. User interactions and operational scenarios were first analyzed to capture what the system must provide from a stakeholder perspective, forming the basis for the SURs.

These user requirements were then refined into SFRs by identifying the system functions required to support the expected behaviors and interactions. In parallel, SNFRs were defined to capture performance, reliability, safety, and other quality constraints applicable to the system.

For clarity and traceability, all requirements are maintained in a separate Excel requirements register for easy access and review. The same requirements are also modeled within the Capella model to ensure consistency and support traceability across operational, functional, and architectural levels.

3.2 System Architecture



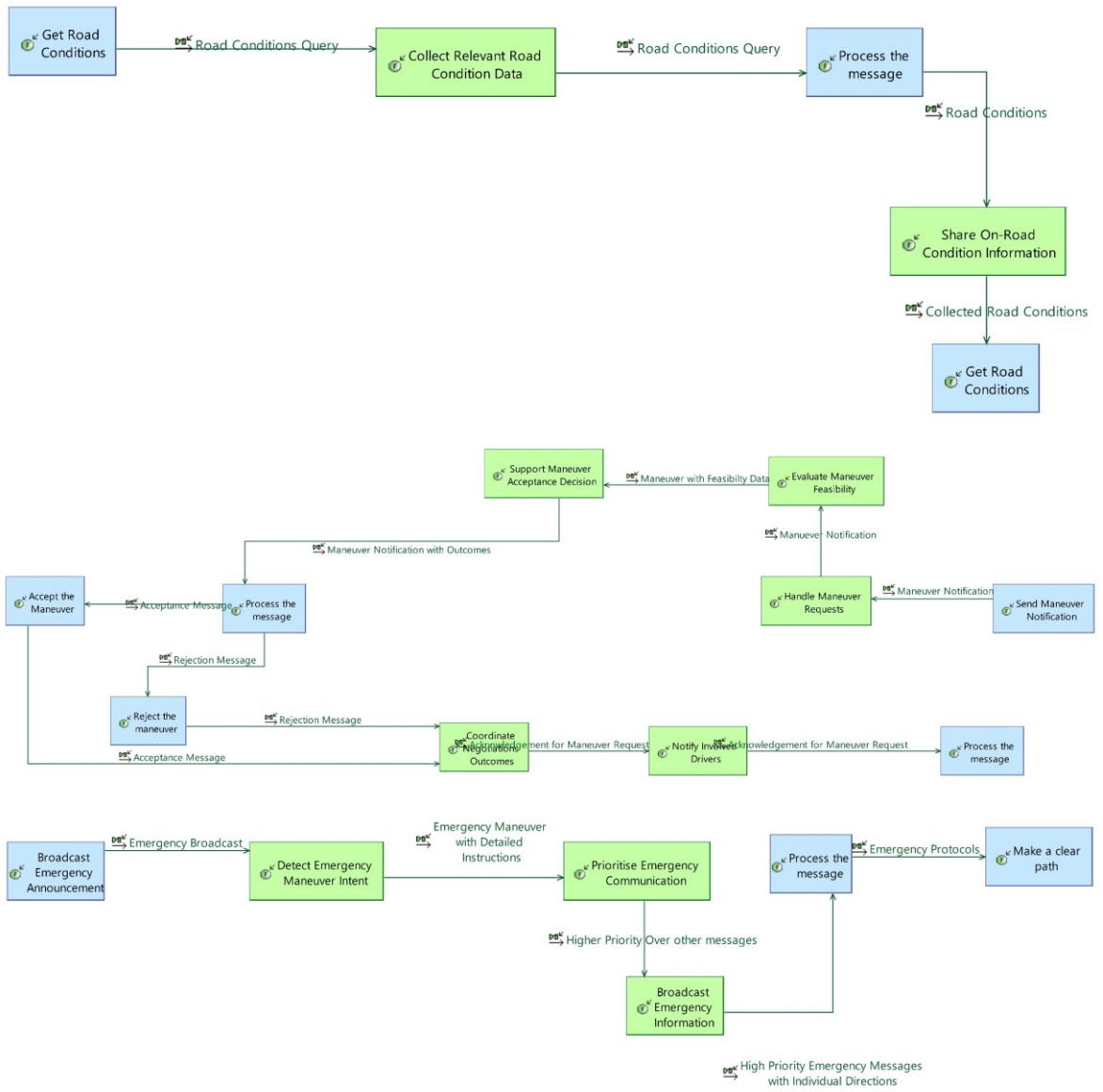
The figure represents a functional architecture view of the Driver-to-Driver Communication System, modeled using MBSE principles. The system is decomposed into interacting system functions that realize maneuver coordination and emergency communication across defined operational actors (casual drivers, involved drivers, and emergency vehicle drivers).

Maneuver management is realized through a sequence of functions including process maneuver request, support maneuver acceptance decision, evaluate maneuver feasibility, and handle maneuver request. These functions exchange functional data flows such as maneuver notifications, feasibility data, acceptance or rejection messages, and acknowledgements, ensuring consistent coordination among involved drivers.

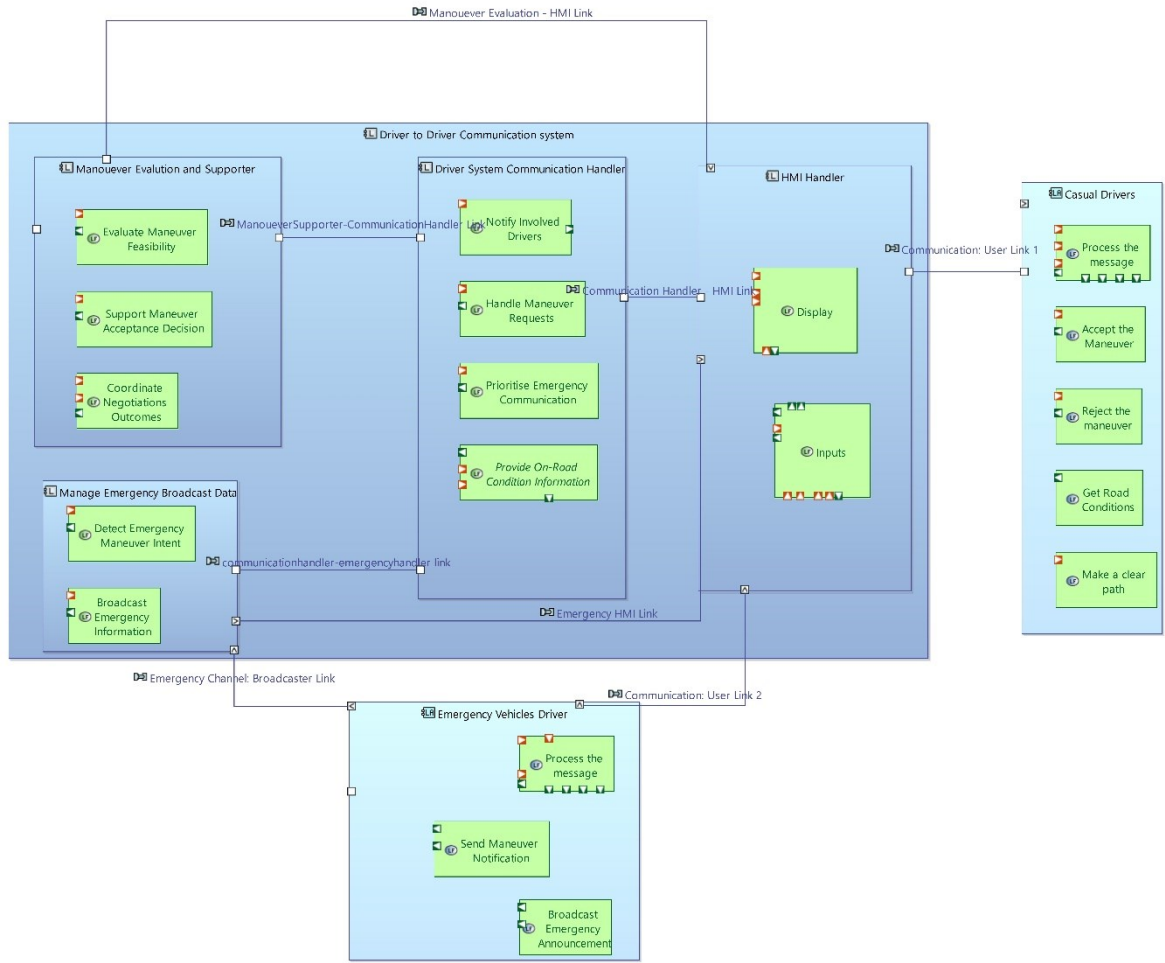
The architecture also incorporates context-awareness functions, such as road condition querying, collection, and sharing, which provide supporting information for maneuver evaluation and negotiation.

A dedicated emergency communication functional chain is modeled, in which emergency maneuver intent is detected, emergency messages are prioritized over nominal traffic communications, and high-priority emergency information is broadcast to affected drivers. This ensures timely path clearance and compliance with emergency protocols.

Overall, the model demonstrates a functionally driven decomposition with clear data exchanges and prioritization mechanisms, supporting both nominal maneuver negotiation and safety-critical emergency operations within a coherent MBSE framework.



4. Logical Architecture



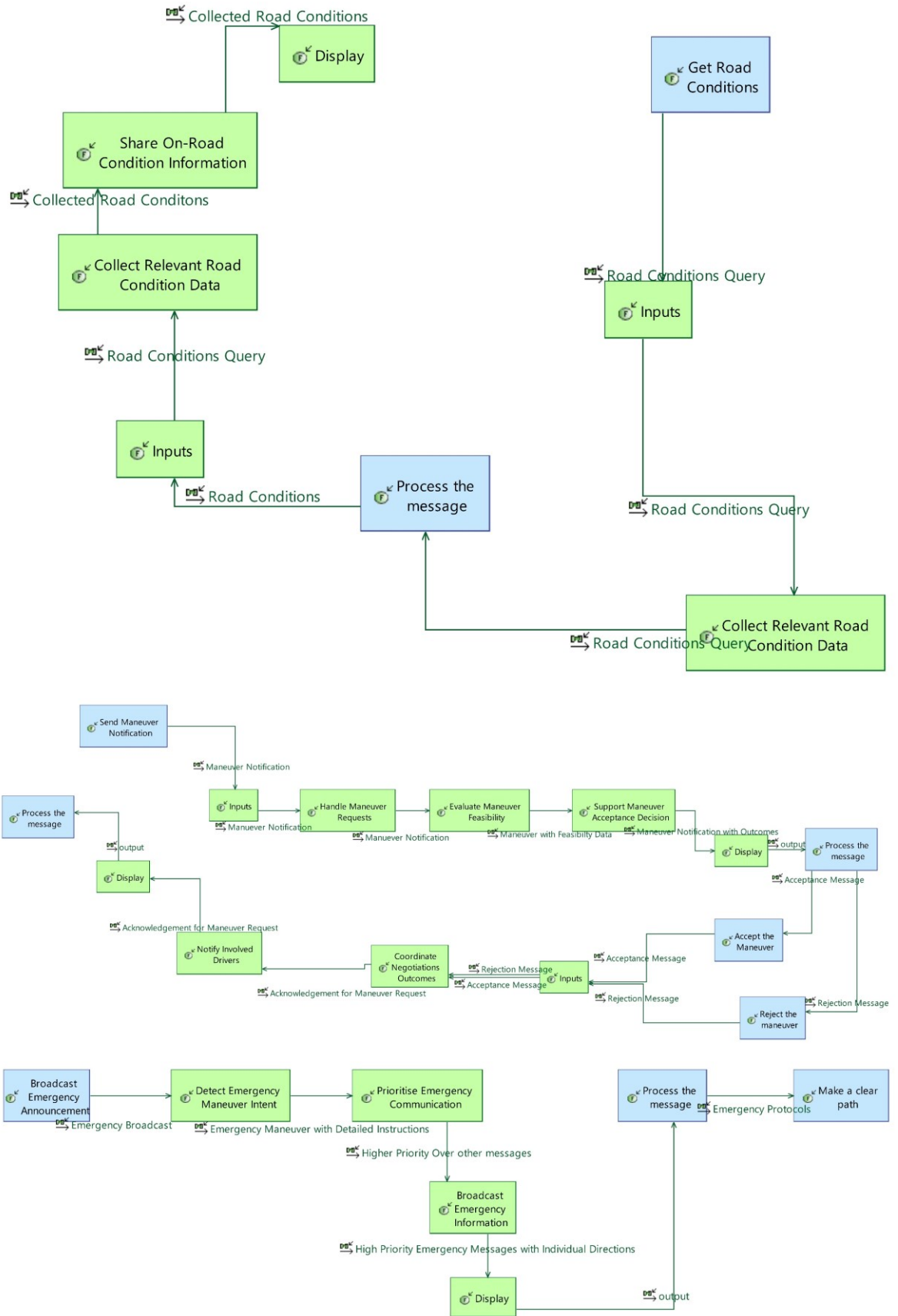
The figure illustrates the logical architecture of the Driver-to-Driver Communication System, describing how system responsibilities are allocated to logical components and how they interact through defined interfaces.

The architecture is centered around the Driver System Communication Handler, which acts as the primary coordination element between maneuver evaluation, communication management, and the human-machine interface (HMI). Maneuver-related inputs originating from drivers are processed through logical functions responsible for maneuver feasibility evaluation, decision support, and negotiation outcome coordination. Based on these evaluations, acceptance or rejection decisions are generated and communicated to the involved drivers via the HMI handler.

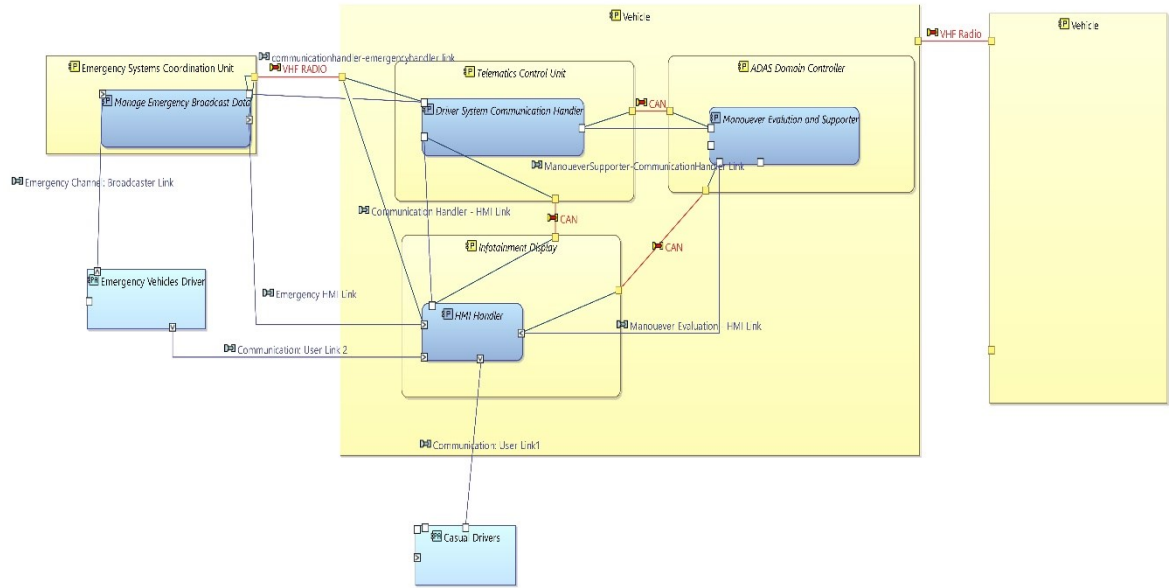
Logical separation is maintained between nominal maneuver negotiation and emergency handling. Emergency-related functionality is realized through dedicated logical components responsible for emergency intent detection, prioritisation of emergency communications, and broadcast of emergency information. Emergency messages are explicitly given higher priority over standard maneuver communications to ensure timely dissemination and driver response.

The architecture also includes supporting information management, where road condition data is queried, collected, and disseminated to relevant logical components to support informed maneuver decisions. All user interactions are mediated through the HMI handler, ensuring controlled and consistent exchange between system logic and external actors.

Overall, the logical architecture demonstrates a clear separation of concerns, explicit communication pathways, and prioritisation mechanisms, enabling traceability from system functions to logical components while supporting both cooperative driving scenarios and safety-critical emergency operations



5. Physical Architecture



The figure presents the **Physical Architecture** of the Driver-to-Driver Communication System, showing the allocation of logical components to physical components and the realization of interfaces through concrete communication means.

Within the vehicle, the **Telematics Control Unit (TCU)** hosts the *Driver System Communication Handler*, acting as the primary gateway for external driver-to-driver and emergency communications. The **ADAS Domain Controller** realizes the *Maneuver Evaluation and Supporter* functionality, enabling maneuver feasibility assessment and decision support. User interaction and information presentation are handled by the **Infotainment Display**, which hosts the *HMI Handler* and interfaces with drivers via defined user links.

Inter-component communication within the vehicle is realized using the **CAN bus**, supporting exchanges between the TCU, ADAS Domain Controller, and HMI. External emergency communication is supported through **VHF radio links**, connecting the vehicle to the **Emergency Systems Coordination Unit** and enabling broadcast of prioritized emergency information. Dedicated emergency and user

communication links ensure separation between safety-critical emergency traffic and nominal maneuver negotiation data.

Overall, the physical architecture demonstrates a clear mapping from logical elements to deployable vehicle subsystems, with explicit communication technologies selected to support reliability, prioritisation, and separation of concerns in both normal and emergency operating conditions.

6. Conclusion and Future Work

This work demonstrated the application of a model-based systems engineering approach to the development of a Driver-to-Driver Communication system. Starting from stakeholder and system requirements, the architecture was progressively refined across operational, functional, logical, and physical layers using Capella. The resulting models establish clear traceability between requirements, system functions, logical components, and physical realizations, enabling consistent reasoning about system behavior, interfaces, and responsibilities. The logical and physical architectures provide a structured view of maneuver negotiation, emergency prioritisation, and user interaction, while ensuring separation between safety-critical and non-critical communication paths.

While the current architecture captures the essential system structure and interactions, several opportunities exist for further refinement. Future work may include deeper behavioral modeling through detailed state machines and scenarios to validate dynamic system behavior under nominal and emergency conditions. The logical-to-physical allocation can be further refined by introducing performance, safety, and timing constraints to support trade-off analysis. In addition, integration of cybersecurity considerations, fault tolerance mechanisms, and scalability aspects would strengthen the robustness of the architecture. Finally, extending the model to support simulation and verification activities would enable early validation of architectural decisions and support transition toward implementation-level design.